

## Trabalho Final - MedicalField

A partir dos conceitos de microsserviços, arquitetura de sistemas e integração de serviços, vamos realizar uma implementação com base no tema a seguir. Primeiro, faremos o mapeamento da solução, em seguida, a implementação.

### Projeto: Prontuário Médico de Emergência (MedicalField)

#### 1. Contexto do Problema

Equipes de resgate e socorristas (como equipes do SAMU, ambulâncias de rodovias ou equipes em cenários de desastres naturais) prestam atendimento pré-hospitalar em locais onde a conectividade com a internet é frequentemente instável, lenta ou totalmente inexistente. Durante um atendimento crítico, dados como **sinais vitais, nível de consciência (Escala de Glasgow), protocolos de triagem (Protocolo de Manchester e START) e medicamentos administrados** precisam ser registrados imediatamente para evitar erros de medicação (ex.: dosagem excessiva ou reações adversas) e garantir a continuidade da assistência quando o paciente for entregue à emergência hospitalar.

Se o sistema depender exclusivamente de uma conexão com a nuvem, a perda de sinal impedirá o socorrista de registrar procedimentos ou consultar regras de segurança clínica. Portanto, a arquitetura do **MedicalField** deve seguir a abordagem *Offline-First*: o dispositivo móvel da ambulância/socorrista deve permitir operação local irrestrita e, assim que o sinal for restabelecido (ou ao chegar à recepção do hospital via Wi-Fi), deve sincronizar as fichas de atendimento com o servidor central, garantindo **idempotência, rastreabilidade auditável e tratamento de conflitos de dados**.

Para refletir a diversidade tecnológica dos ambientes corporativos de saúde, o ecossistema backend deve ser composto por **pelo menos três microsserviços em linguagens de programação distintas**, somado a uma **interface gráfica administrativa**, acompanhado de uma aplicação cliente, também com **interface gráfica (GUI)**.

Existem transações operacionais adicionais importantes para o fluxo do dia a dia. A plataforma deve possibilitar gerar a Ficha de Atendimento em formato PDF para download, bem como, realizar o envio do arquivo anexado por e-mail para algum destinatário: paciente ou familiar.

## 2. Especificação dos Requisitos

### 2.1 Requisitos Gerais e Arquiteturais

- **Multi-Linguagem Backend:** O ecossistema backend deve possuir **no mínimo três serviços/APIs autônomos implementados em linguagens de programação diferentes**.
- **Persistência Dual (Banco de Dados):**
  - *Servidor Central/Nuvem:* Banco de dados relacional ou NoSQL de livre escolha para consolidar os prontuários e auditorias.
  - *Cliente Local (GUI):* Banco de dados local embarcado no cliente para suportar gravações *offline*.
- **Ambiente de Implantação:** Execução obrigatória na **Máquina Virtual (VM) da Univates**. Não é necessário utilizar contêineres.
- **Segurança e Autenticação:** Todas as rotas (com exceção da rota de login) exigem autenticação associado às credenciais do profissional de saúde (CRM, COREN ou ID de Socorrista).
- **Auditoria e Logs de Acesso (LGPD / Regulamentações de Saúde):** Todos os serviços backend devem conter um *middleware/interceptor* que registre logs detalhados de acesso a cada requisição HTTP, como: *Data\_hora*, *IP do Cliente*, *ID do Profissional Autenticado*, *Método HTTP*, *Rota*, *Status Code* e *Tempo de Processamento (ms)*.
- **Interface Gráfica com Simulador de Rede:** A aplicação cliente deve fornecer interface gráfica para preenchimento da ficha de atendimento e um botão/chave seletora para **"Simular Conectividade (Online / Offline)"**, permitindo a execução do roteiro de testes em sala de aula.
- **Interface Gráfica da área Administrativa:** A aplicação que executa no servidor em nuvem necessita ter uma interface gráfica que possibilite: autenticar, visualizar os registros de atendimento e visualizar cadastros de apoio. Inserções poderão ser realizadas via banco de dados manualmente.

### 2.2 Padrões de Projeto (Design Patterns) - Sugestão

1. **Repository Pattern:** Isolamento da camada de dados locais (banco embarcado) e remotos (API REST) das regras da interface gráfica.
2. **Strategy Pattern:** Encapsulamento dos algoritmos de resolução de concorrência e mesclagem (*merge*) das fichas médicas durante a sincronização (ex.: priorização dos dados gravados com o carimbo de data/hora mais recente pelo profissional de saúde).
3. **Observer Pattern:** Monitoramento do estado do simulador de rede para acionamento automático da fila de sincronização pendente (*Outbox*) assim que a conexão retornar.
4. **Outros 3 padrões de livre escolha.**

## 2.3 Especificação das APIs

### Endpoints sugestão

| Método | Endpoint          | Autenticada | Ação                                             |
|--------|-------------------|-------------|--------------------------------------------------|
| POST   | /auth/login       | Não         | Autentica o profissional e retorna token válido. |
| GET    | /medicamento      | Sim         | Obtém todos os medicamentos.                     |
| GET    | /prontuario       | Sim         | Obtém todos prontuários de todas as pessoas.     |
| GET    | /prontuario/{id}  | Sim         | Obtém o prontuário de uma pessoa.                |
| POST   | /pessoa           | Sim         | Cadastra nova pessoa.                            |
| GET    | /atendimento/{id} | Sim         | Obtém todos os atendimentos de uma pessoa.       |
| POST   | /prontuario       | Sim         | Cadastra um prontuário.                          |
| POST   | /atendimento      | Sim         | Cadastra um atendimento.                         |
| POST   | /log              | Sim         | Envia dados de locais de log do offline.         |

Contemplar rotas para envio do e-mail e geração do documento PDF, dentre outras rotas conforme a necessidade.

### Checklist do projeto

No dia da apresentação deve ser entregue e apresentado:

1. Modelo de arquitetura: desenho + tecnologias utilizadas
2. Documentação das APIs, seus endpoints e modelagem dos Bancos de Dados
3. Roteiro de testes

## Roteiro de testes

1. Portal: Autenticar
2. Portal: Listar cadastros
3. Portal: Listar prontuários
4. Portal: Listar atendimentos
5. Cliente: Autenticar
6. Cliente: Sincronizar dados a partir do Portal (receber dados)
7. Cliente: Online - Registrar um atendimento
8. Portal: Listar atendimentos - visualizar atendimento registrado
9. Portal: Inserir novos medicamentos via Banco de Dados
10. Portal: Listar atendimentos
11. Cliente: Sincronizar dados a partir do Portal (receber dados)
12. Cliente: Offline - Registrar ocorrência de socorro: Criar pessoa, Criar prontuário, Registrar atendimento
13. Portal: Listar prontuários e atendimentos: cadastro novo não deve ser exibido
14. Cliente: Online - Sincronizar dados para o Portal
15. Portal: Listar prontuários e atendimentos: cadastro novo deve ser exibido
16. Demonstrar geração da Ficha de atendimento
17. Demonstrar e-mail enviado com Ficha de atendimento em anexo

## Requisitos técnicos

- Banco de dados: livre escolha
- Linguagens de programação: livre escolha
- Deve ser utilizada mais de uma linguagem de programação nos serviços (APIs)
- A solução deve ser executada na Máquina Virtual da Univates
- As rotas devem ser autenticadas
- Devem ser gerados logs dos acessos de cada rota
- Deve ser possível validar as rotas via Postman, Insomnia ou similar
- A aplicação deve possuir interface gráfica para validar as ações do roteiro de testes